Contents


```
% MAE 283B Final Project
% Charlie Frank
% Spring 2026
clear
close all
clc
```

Question 1

w/ Load data from EXOTHERMIC PROCESS PLANT

```
load("mae283bexofiles.mat")
% Question 1
C = ss(Ac, Bc, Cc, Dc, 1);
Ts = 1;
data = iddata(y(:), u(:), Ts);
% 1.A: explanation
fprintf('\n1.A Explanation of choices for Parametrization\n');
fprintf(['\n For the parametrization of G^, an ARX model structure was \n' ...
    'chosen with a 7th order single step delay. ARX places no stability \n' ...
    'constraints on the estimated plant, it allows the identified model to\n' ...
    ' be unstable as OKed in the problem statement via poles outside the\n' ...
    'unit circle. The noise model H^ is defined in ARX as 1/A(q) which is \n' ...
    'useful for direct ID of CL data. Model order was 7 to utilize maximum \n' ...
    'allowed order while capturing dominant dynamics of unstable plant.\n']);

% 1.B stuff
na = 7;
nb = 7;
nk = 1;
Ghat_id = arx(data, [na nb nk]);
Ghat = tf(Ghat_id);
fprintf('\n1.B Pole Locations of Estimated Ghat(q)\n');
plant_poles = pole(Ghat);
disp(plant_poles);

% 1.C
CL = feedback(Ghat, C);
fprintf('\n1.C Pole Locations of the Closed-Loop System\n');
cl_poles = eig(CL);
disp(cl_poles);
```

```
1.A Explanation of choices for Parametrization

 For the parametrization of G^, an ARX model structure was
chosen with a 7th order single step delay. ARX places no stability
constraints on the estimated plant, it allows the identified model to
 be unstable as OKed in the problem statement via poles outside the
unit circle. The noise model H^ is defined in ARX as 1/A(q) which is
useful for direct ID of CL data. Model order was 7 to utilize maximum
allowed order while capturing dominant dynamics of unstable plant.

1.B Pole Locations of Estimated Ghat(q)
  -0.7137 + 0.0000i
  -0.3627 + 0.5860i
  -0.3627 - 0.5860i
   1.0326 + 0.0000i
   0.5164 + 0.3039i
   0.5164 - 0.3039i
   0.2640 + 0.0000i


1.C Pole Locations of the Closed-Loop System
   0.5447 + 0.7727i
   0.5447 - 0.7727i
  -0.0583 + 0.8884i
  -0.0583 - 0.8884i
  -0.6270 + 0.5622i
  -0.6270 - 0.5622i
  -0.8262 + 0.0000i
  -0.3231 + 0.5911i
  -0.3231 - 0.5911i
  -0.5315 + 0.0000i
   0.8672 + 0.1191i
   0.8672 - 0.1191i
   0.4759 + 0.0000i
   0.2319 + 0.0000i
```

Question 2

2.A Explanation

```
fprintf('\n2.A Explanation of choices of Methodology\n');
fprintf(['\n The 2 stage method was chosen for CL ID. This method is \n' ...
    'appropriate because G0 and C are unstable. Stage 1 regresses u onto \n' ...
    'r1 and r2 using an ARX model, removing the noise correlated with the \n' ...
    'output and retaining the excitable portion. Stage 2 IDs G^ using ARX \n' ...
    'again, since u^ is uncorrelated with the noise on y, second stage \n' ...
    'ARX makes sense and seeminlgy works. These methods are well suited \n' ...
    'to the unstable nature of G0 and C.\n']);


Ts = 1;
t = (1:length(y))';

% Stage 1: Predict u(t) from external references
data_stage1 = iddata(u(:), [r1(:), r2(:)], Ts);
```

```
sys_u = arx(data_stage1, [14 [14, 14] [1, 1]]);
u_hat = predict(sys_u, data_stage1);

% Stage 2: Identify Ghat(q) from projected input
data_stage2 = iddata(y(:), u_hat.OutputData, Ts);
Ghat_cl_id = arx(data_stage2, [7 7 1]);
Ghat_cl = tf(Ghat_cl_id);

% 2.B poles
fprintf('\n2.B Pole Locations of Estimated Ghat(q) (Two-Stage)\n');
plant_poles_cl = pole(Ghat_cl);
disp(plant_poles_cl);

% 2.C poles CL
CL_cl = feedback(Ghat_cl, C);
fprintf('\n2.C Pole Locations of the Closed-Loop System (Two-Stage)\n');
cl_poles_cl = eig(CL_cl);
disp(cl_poles_cl);

% 2.d Closed-Loop Simulation and Plotting
G_r1 = feedback(Ghat_cl, C);
T = feedback(Ghat_cl * C, 1);
y_hat_total = lsim(G_r1, r1(:), t) + lsim(T, r2(:), t);

fprintf('\n2.D Figure 1\n')
% 2.D: Figure 1: plot of measured y versus y^
figure(1);clf;
plot(t, y(:), 'b', 'LineWidth', 0.7); hold on;
plot(t, y_hat_total, 'r--', 'LineWidth', 0.7);
grid on; xlim([-50, 5050]); ylim([-80, 80]);
xlabel('Time Index (t)'); ylabel('Output y(t)');
title(sprintf('Figure 1: Measured vs. Simulated Closed-Loop Output'));
legend('Measured y(t)', 'Simulated y_hat(t) (Two-Stage Method)', 'Location', 'best');
```

2.A Explanation of choices of Methodology

The 2 stage method was chosen for CL ID. This method is appropriate because G0 and C are unstable. Stage 1 regresses u onto r1 and r2 using an ARX model, removing the noise correlated with the output and retaining the excitable portion. Stage 2 IDs G^ using ARX again, since u^ is uncorrelated with the noise on y, second stage ARX makes sense and seeminlgy works. These methods are well suited to the unstable nature of G0 and C.

2.B Pole Locations of Estimated Ghat(q) (Two-Stage)

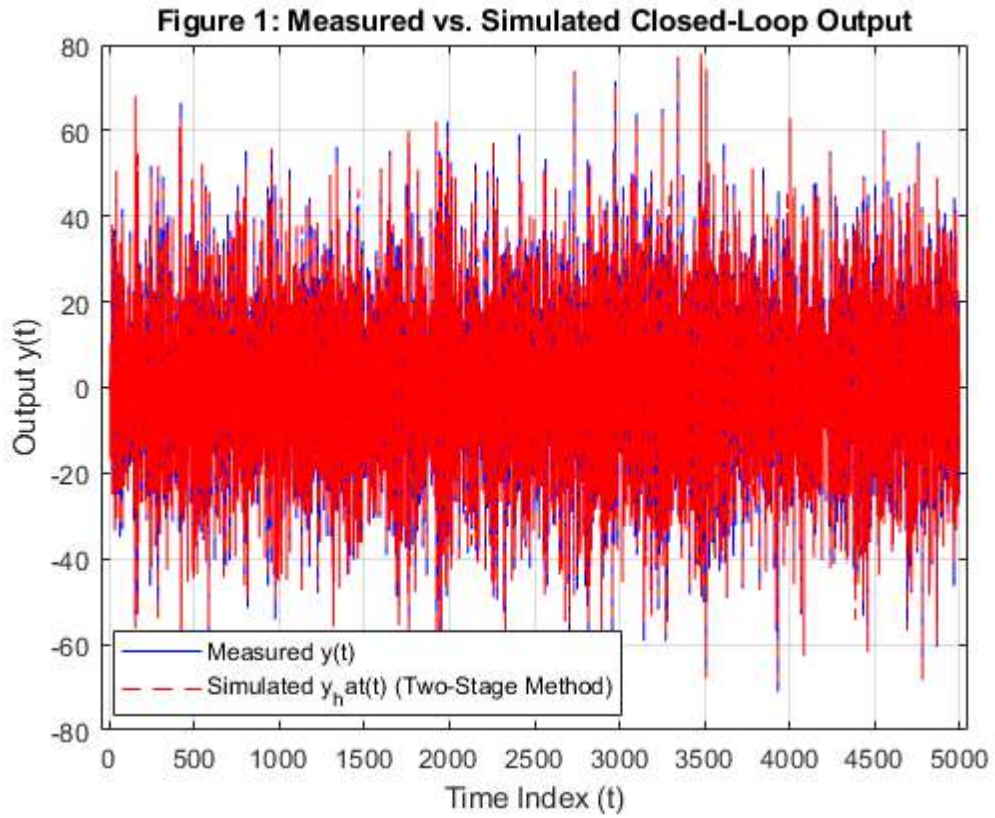
```
   0.9314 + 0.0000i
  -0.7085 + 0.0000i
   0.3134 + 0.4544i
   0.3134 - 0.4544i
  -0.2164 + 0.3678i
  -0.2164 - 0.3678i
  -0.1152 + 0.0000i
```

2.C Pole Locations of the Closed-Loop System (Two-Stage)

```
   0.5438 + 0.7789i
   0.5438 - 0.7789i
```

```
   0.8954 + 0.0000i
   0.6714 + 0.0000i
  -0.0530 + 0.8694i
  -0.0530 - 0.8694i
  -0.6382 + 0.5721i
  -0.6382 - 0.5721i
  -0.0721 + 0.6618i
  -0.0721 - 0.6618i
  -0.8090 + 0.0000i
  -0.5925 + 0.0000i
  -0.3897 + 0.0000i
   0.2318 + 0.0000i
```

2.D Figure 1



Question 3

w/ load data from MICROGRID

```
clear
load("mae283bmicronocontrfiles.mat")
% Question 3
Ts = 1;
t = (1:length(y))';
N = length(u);
data_ol = iddata(y(:), u(:), Ts);

% 3.A: explanation
fprintf('\n3.A Explanation of choices for Parametrization\n');
fprintf(['\n An OE model structure was chosen with order 8 and delay of \n' ...
```

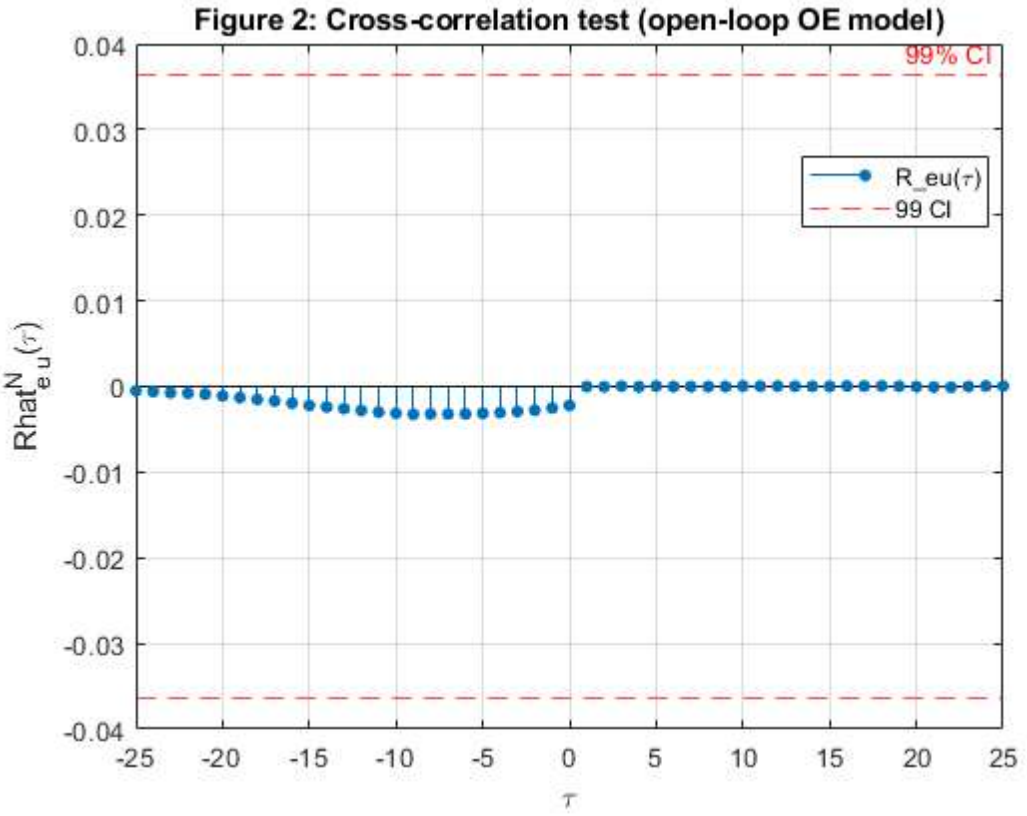
```
    '1 step. The microgrid is stable and OE has no stability constraints, \n' ...
    'stable poles are more easy to recover. OE also seperates the plant \n' ...
    'model from the noise model, allowing the PE to not distort the plant \n' ...
    'estimate via the noise estimate. 8th order was used to capture \n' ...
    'oscillatory dynamics of power flow. \n']);

% Model Estimation
nb = 8; nf = 8; nk = 1;
Ghat_ol_id = oe(data_ol, [nb nf nk]);
Ghat_ol = tf(Ghat_ol_id);

maxlag = 25;
alpha  = 2.576 / sqrt(N);   % 99 % confidence bound  (z_{0.005})

% Prediction error: for OE,  epsilon = y - G(q,th)*u
%yhat = predict(Ghat_ol, data_ol);
%eps  = data_ol.y - yhat.y;
Ghat_ol_tf = tf(Ghat_ol_id);
yhat_sim = lsim(Ghat_ol_tf, u(:), t);
eps = y(:) - yhat_sim;

[Reu, lags] = xcorr(eps, u, maxlag, 'biased');

% 3.B: Figure 2: Cross Correlation test
fprintf('\n3.B Figure 2\n')
figure(2); clf;
stem(lags, Reu, 'filled', 'MarkerSize', 4); hold on;
yline( alpha, 'r--', '99% CI');
yline(-alpha, 'r--');
xlabel('\tau'); ylabel('{Rhat}^N_{e u}(\tau)');
title('Figure 2: Cross-correlation test (open-loop OE model)');
xlim([-maxlag maxlag]);
legend('R\_eu(\tau)','99 CI', 'Location','best');
grid on;


% 3.C: Figure 3: Step Response of Estimated Model Ghat(q)
fprintf('\n3.C Figure 3\n')
figure(3);clf;
step(Ghat_ol, 200);
title('Figure 3: Step Response of Estimated Model Ĝ(q)');
grid on;
```

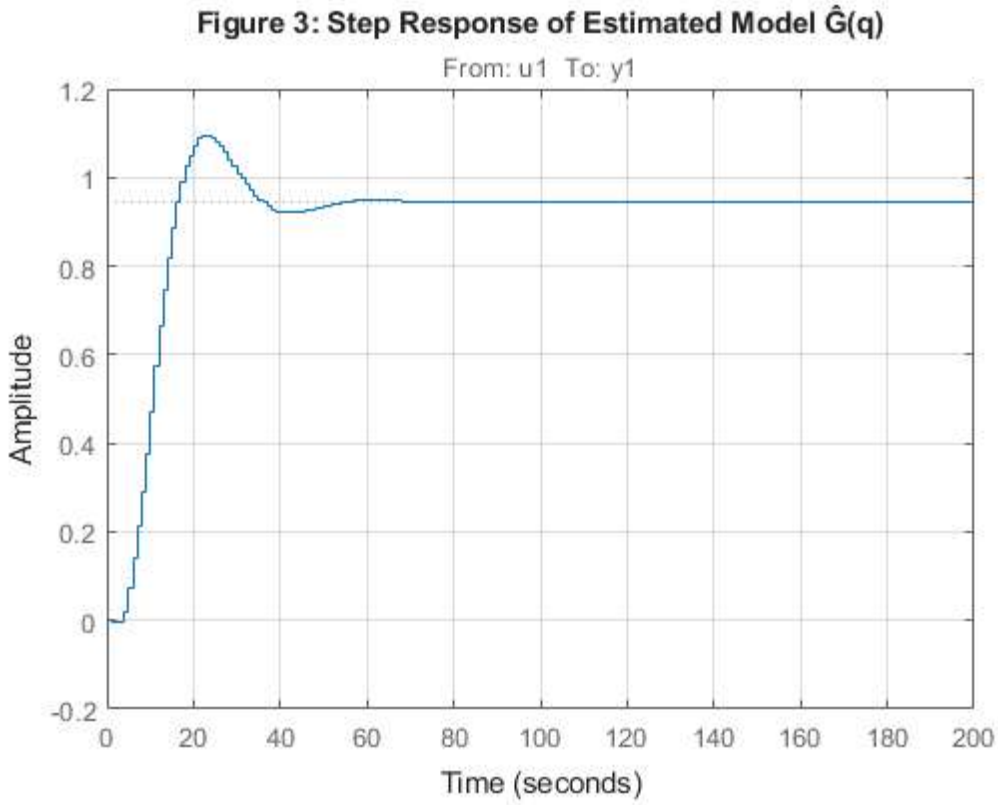
3.A Explanation of choices for Parametrization

An OE model structure was chosen with order 8 and delay of
1 step. The microgrid is stable and OE has no stability constraints,
stable poles are more easy to recover. OE also seperates the plant
model from the noise model, allowing the PE to not distort the plant
estimate via the noise estimate. 8th order was used to capture
oscillatory dynamics of power flow.

3.B Figure 2

3.C Figure 3





Question 4

w/ load data from MICROGRID W/ Controller

```
clear
load("mae283bmicrofiles.mat")
Ts = 0.01;
N_cl = length(u);    % closed-loop dataset length
C_ss  = ss(Ac, Bc, Cc, Dc, Ts);
C_tf  = tf(C_ss);

% Sensitivity function S(q) = 1 / (1 + C(q)*G0(q))
% STAGE 1
data_cl   = iddata(u, r1, Ts); % output = u,  input = r1
S_order   = 10;
S_hat     = oe(data_cl, [S_order S_order 1]);

% Noise free input estimate
unf_data  = sim(S_hat, iddata([], r1, Ts));
u_nf      = unf_data.y;

% 4.A: Figure 4: measured u(t) vs simulated u_nf(t)
fprintf('\n4.A Figure 4\n')
figure(4); clf;
t_ax = (0:N_cl-1).' * Ts;
plot(t_ax, u, 'b', t_ax, u_nf, 'r--', 'LineWidth', 1);
xlabel('Time (s)'); ylabel('u');
title('4.A: Figure 4: 4.A Closed-loop input: measured u(t) vs {u}\^_{nf}(t)');
legend('u(t)', 'u_{nf}(t)', 'Location','best');
grid on;

% STAGE 2: Identify G(q)
data_s2  = iddata(y, u_nf, Ts);
nb2 = 5; nf2 = 5; nk2 = 1;        % 5th order as specified
Ghat_cl  = oe(data_s2, [nb2 nf2 nk2]);

% 4.B: Figure 5:  measured y(t) vs simulated y_hat(t) = G_hat(q)*u_nf(t)
yhat_cl_data = sim(Ghat_cl, iddata([], u_nf, Ts));
y_hat_cl     = yhat_cl_data.y;
fprintf('\n4.B Figure 5\n')
figure(5); clf;
plot(t_ax, y, 'b', t_ax, y_hat_cl, 'r--', 'LineWidth', 1);
xlabel('Time (s)'); ylabel('y');
title('Figure 5 – 4.B Closed-loop output: measured y(t) vs {y}\^(t)');
legend('y(t)','${y}\^(t)$','Location','best');
grid on;

% 4.C: Figure 6: Step response of closed-loop identified G_hat
fprintf('\n4.C Figure 6\n')
figure(6); clf;
step(d2c(tf(Ghat_cl)));
title('Figure 6 – 4.C Step response of {G}\^(q) (two-stage CL identification)');
ylabel('Amplitude'); xlabel('Time (s)');
grid on;

%  4.D: Closed-loop pole locations of negative feedback
G_tf   = tf(Ghat_cl);
CL_tf  = feedback(G_tf * C_tf, 1);  % negative unity feedback
cl_poles = pole(CL_tf);
fprintf('\n4.D: Closed-loop poles of G_hat(q)*C(q):\n');
disp(cl_poles);
```

4.A Figure 4

4.B Figure 5

4.C Figure 6

4.D: Closed-loop poles of G_hat(q)*C(q):
  -0.1660 + 0.5620i
  -0.1660 - 0.5620i
   1.0157 + 0.0000i
   1.0134 + 0.0842i
   1.0134 - 0.0842i
   0.9325 + 0.1135i
   0.9325 - 0.1135i
   0.1193 + 0.0000i

4.A: Figure 4: 4.A Closed-loop input: measured u(t) vs u^$_{nf}$(t)

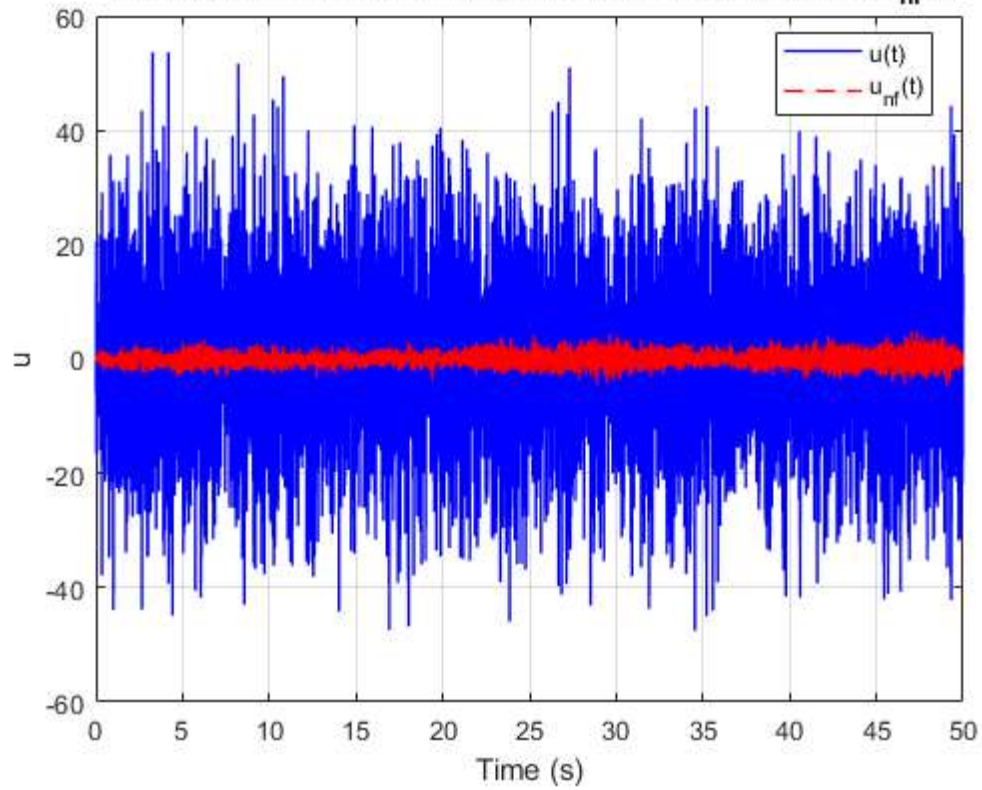


Figure 5 – 4.B Closed-loop output: measured y(t) vs y^(t)

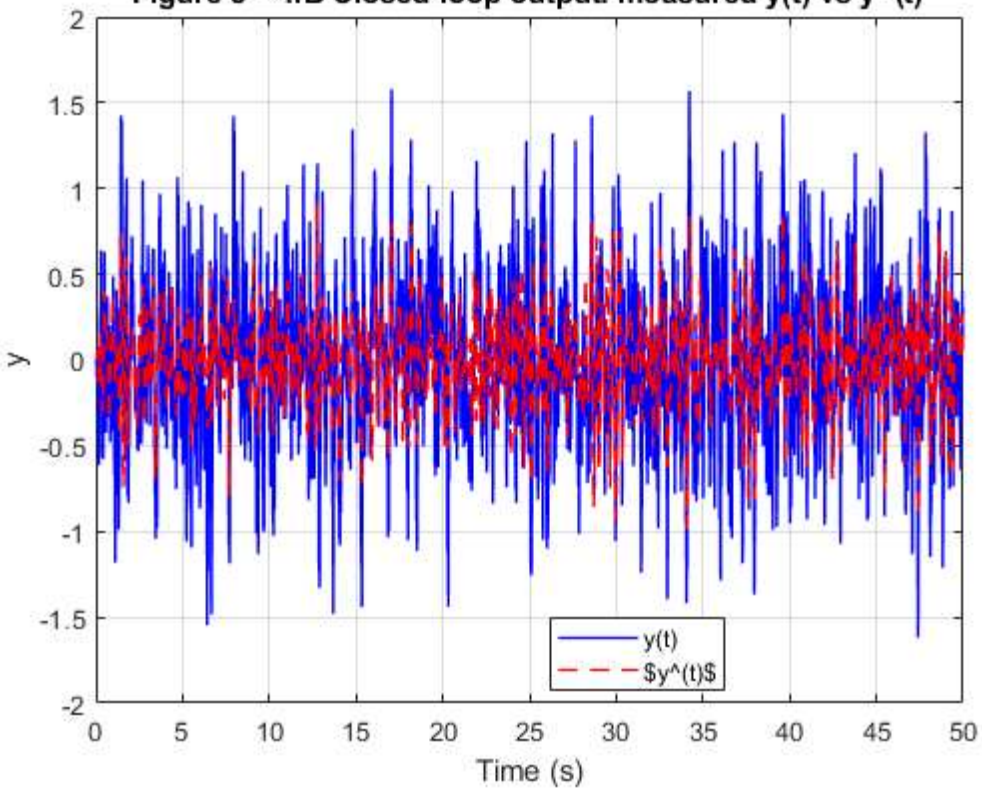
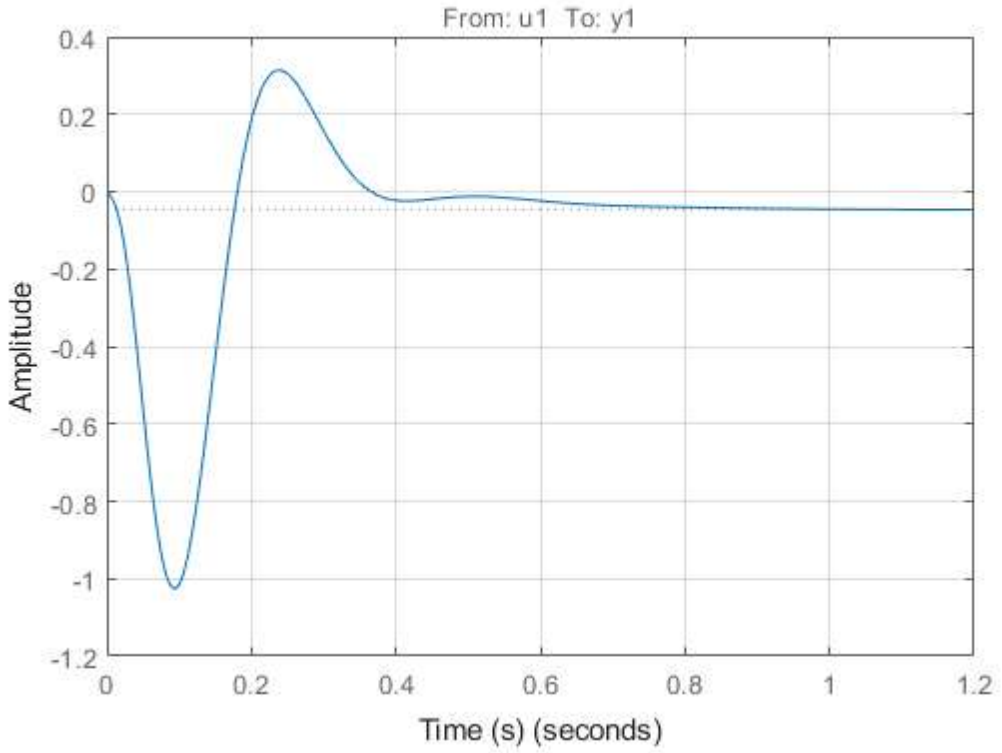


Figure 6 – 4.C Step response of G^(q) (two-stage CL identification)



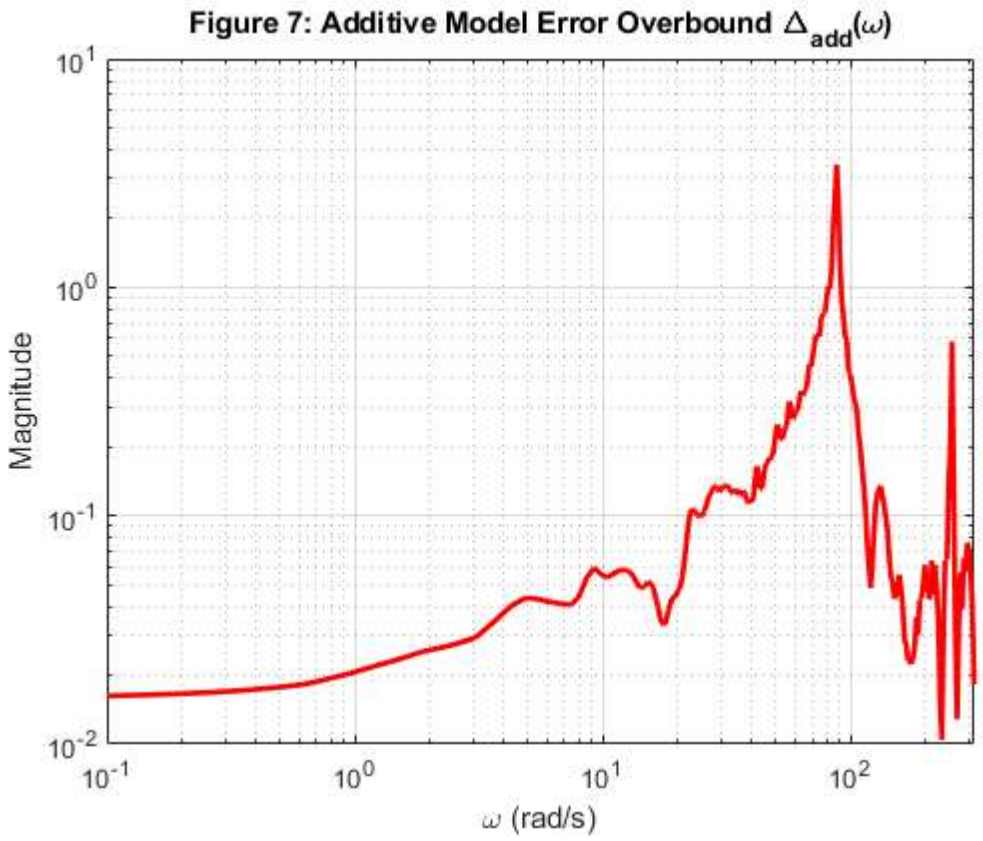
Quesiton 5

methods based on lecture 17 content

```
z = y - y_hat_cl; % MEM output
x = u_nf;  % MEM input
N_data = length(z);
w = logspace(-1, log10(pi/Ts), 512).';

% High order ARX for delta(q)
n_mem = 20;
data_z = iddata(z, x, Ts);
Delta_arx = arx(data_z, [n_mem n_mem 1]);

% |delta(e^{jw})|
[mag_D, ~, ~] = bode(tf(Delta_arx), w);
Delta_hat_mag = squeeze(mag_D);

% Spectra Φ_x and Φ_n
[Pxx, w_sp] = pwelch(x, hanning(512), 256, 1024, 1/Ts, 'onesided');
Phi_x = interp1(w_sp*2*pi, Pxx, w, 'linear', 'extrap');
Phi_x = max(Phi_x, 1e-12);

eps_vec = resid(Delta_arx, data_z).y;
[Pnn, w_sp2] = pwelch(eps_vec, hanning(512), 256, 1024, 1/Ts, 'onesided');
Phi_n = interp1(w_sp2*2*pi, Pnn, w, 'linear', 'extrap');
Phi_n = max(Phi_n, 1e-12);

% Covariance approximation
Cov_Delta = (n_mem / N_data) * (Phi_n ./ Phi_x);
```

```
% Overbound Δ_add(ω)
Delta_add = Delta_hat_mag + 3*sqrt(Cov_Delta);

% Figure 7
fprintf('\n5.A Figure 7\n')
figure(7); clf;
loglog(w, Delta_add, 'r', 'LineWidth', 2);
xlabel('\omega (rad/s)');
ylabel('Magnitude');
title('Figure 7: Additive Model Error Overbound \Delta_{add}(\omega)');
grid on;
xlim([w(1) w(end)]);
```

```
5.A Figure 7
```

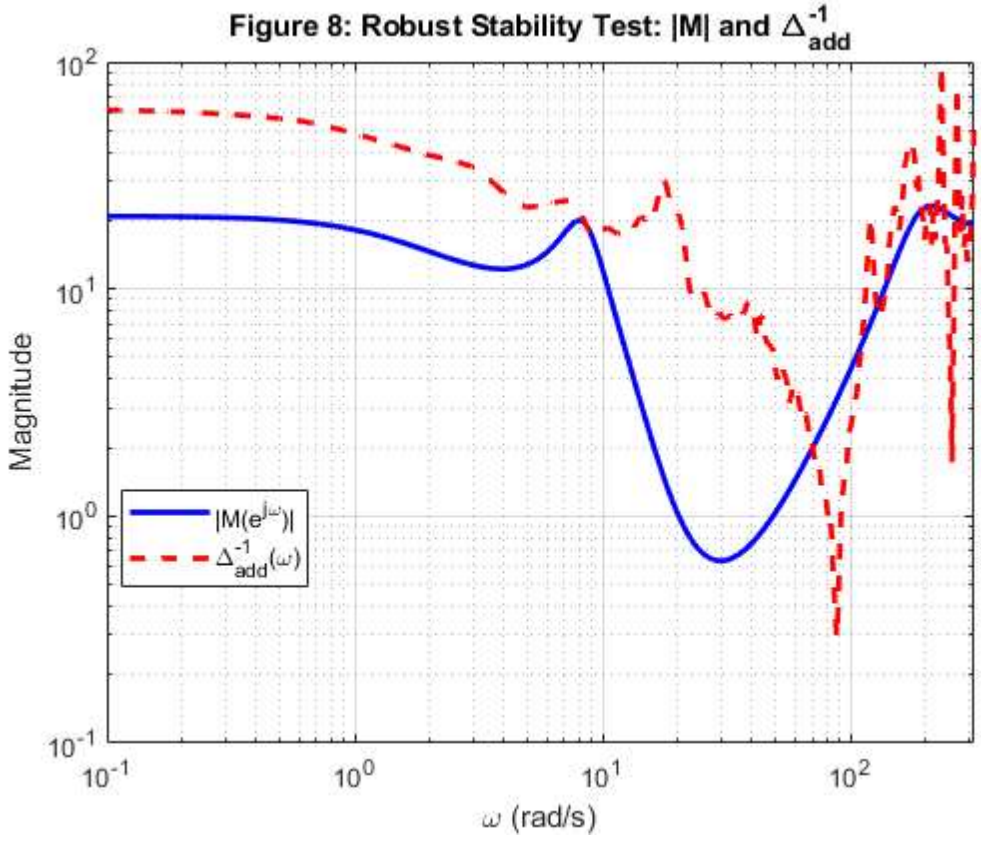


Question 6

```
G_tf = tf(Ghat_cl);
M_tf = C_tf / (1 + G_tf*C_tf);
fprintf('6.A Robust Stability Test: Expression for M(q):\n');
fprintf(' M(q) = C(q) / (1 + G_hat(q)*C(q))\n');

[mag_M, ~, ~] = bode(M_tf, w);
mag_M = squeeze(mag_M);
Delta_add_inv = 1 ./ Delta_add;

% Figure 8
fprintf('\n6.B Figure 2\n')
figure(8); clf;
```

```
loglog(w, mag_M, 'b', 'LineWidth', 2); hold on;
loglog(w, Delta_add_inv, 'r--', 'LineWidth', 2);
xlabel('\omega (rad/s)');
ylabel('Magnitude');
title('Figure 8: Robust Stability Test: |M| and \Delta_{add}^{-1}');
legend('|M(e^{j\omega})|','\Delta_{add}^{-1}(\omega)','Location','best');
grid on;
xlim([w(1) w(end)]);
% Conclusions
fprintf('6.B Robust Stability Test: Conclusions:\n');
fprintf(['Based on the information shown in figure 8 I cannot conclude \n' ...
    'robust stability. The red dashed line, delta_add^-1, does not \n' ...
    'overbound the blue line |M| the entire frequency range. This shows \n' ...
    'that there exists some plant within my uncertainty set that would \n' ...
    'destabilize the loop. \n']);
```

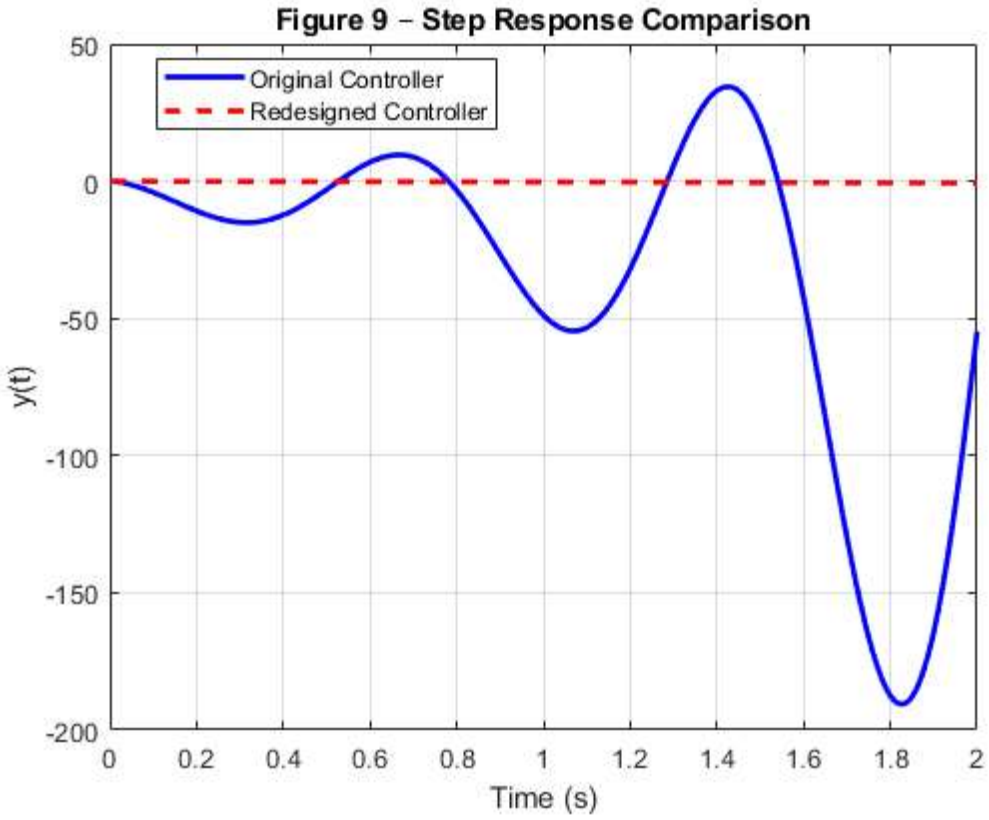
```
6.A Robust Stability Test: Expression for M(q):
 M(q) = C(q) / (1 + G_hat(q)*C(q))

6.B Figure 2
6.B Robust Stability Test: Conclusions:
Based on the information shown in figure 8 I cannot conclude
robust stability. The red dashed line, delta_add^-1, does not
overbound the blue line |M| the entire frequency range. This shows
that there exists some plant within my uncertainty set that would
destabilize the loop.
```



Question 7 ATTEMPT

```
C_old = C_tf;
w_cut = 0.5;

% Discrete first order low pass filter
s = tf('s');
H_lp_continuous = 1 / ( (1/w_cut)*s + 1 );
H_lp = c2d(H_lp_continuous, Ts, 'tustin');

% Apply the filter to the original controller with a mild gain reduction
C_new = 0.7 * C_old * H_lp;

T_old = feedback(G_tf * C_old, 1);
T_new = feedback(G_tf * C_new, 1);

M_new = C_new / (1 + G_tf * C_new);
mag_M_new = squeeze(bode(M_new, w));

figure(9); clf;
t_step = 0:Ts:(200*Ts);

[y_old, t_old] = step(T_old, t_step);
[y_new, t_new] = step(T_new, t_step);

plot(t_old, y_old, 'b', 'LineWidth', 2); hold on;
plot(t_new, y_new, 'r--', 'LineWidth', 2);

xlabel('Time (s)');
ylabel('y(t)');
title('Figure 9 – Step Response Comparison');
legend('Original Controller', 'Redesigned Controller', 'Location','best');
grid on;

fprintf('7:BONUS\n');
fprintf(['Figure 9 shows that the reduction factor and LPF reduce the \n' ...
    'ocsillations that the original C was seeing, this should improve \n' ...
    'the overall controller response. The controller is now stronger at low frequencies ']);
```

```
7:BONUS
Figure 9 shows that the reduction factor and LPF reduce the
ocsillations that the original C was seeing, this should improve
the overall controller response. The controller is now stronger at low frequencies
```



Published with MATLAB® R2024b